CSE221: Algorithms

Depth First Search (DFS)

Prepared by: A B M Muntasir Rahman [MUNR]



Graph Traversal

- **Given a graph G = (V, E), and a source vertex s, we want to visit all the vertices Є V exactly once!**

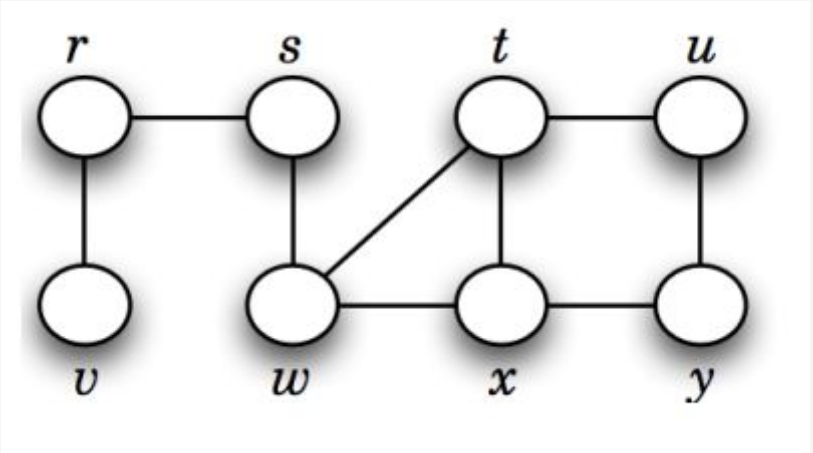
- We will discuss two graph traversal algorithms: **Breadth First Search (BFS) and Depth First Search (DFS)**

Depth First Search (DFS)

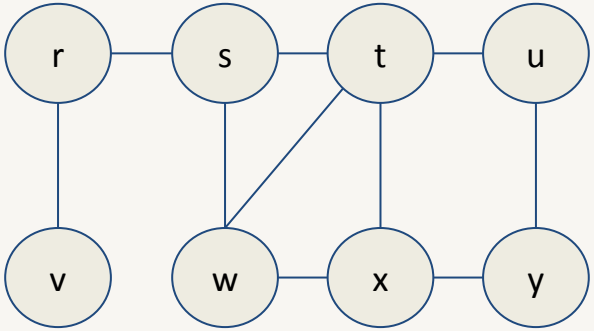
- Traverses the entire graph **depth-wise** without regard for which edge it takes next until it can not go any further at which point it backtracks and continues.
- Uses **Stack** for traversal.

If we start from source vertex s, we can go to w, then to x, then to y, then u, then t. After that, we can backtrack from t to u, u to y, y to x, x to w and then to s. From s we can go deeper to r and then to v.

So, one **DFS sequence** will be,
s, w, x, y, u, t, r, v



Simulation



Stack

Visited

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

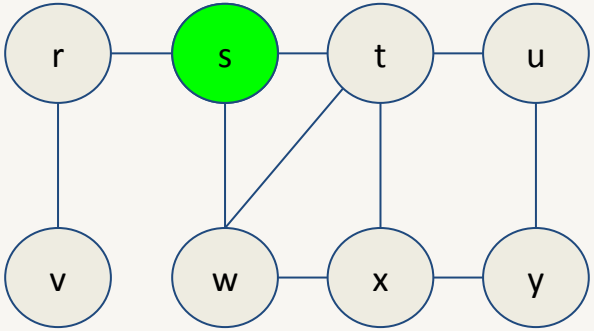
Start Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

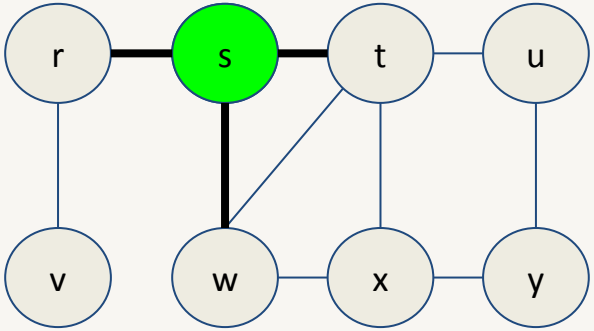
Start Time

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

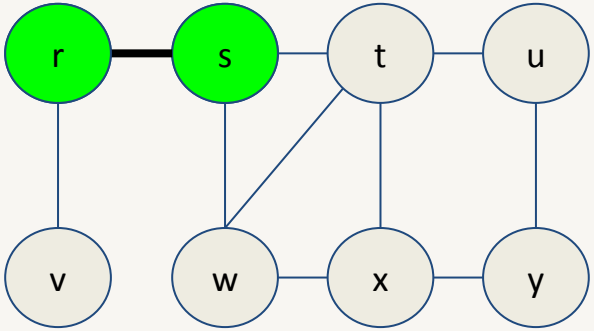
Start Time

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s r

Stack

Visited

1	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

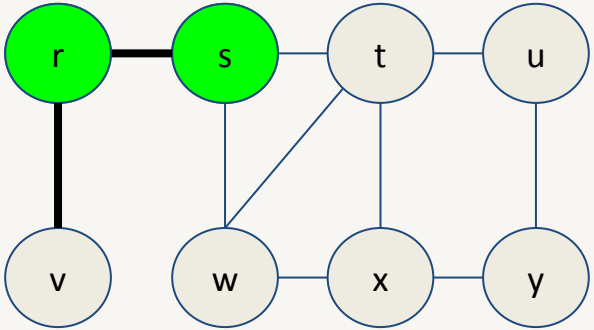
Start Time

2	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s r

Stack

Visited

1	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

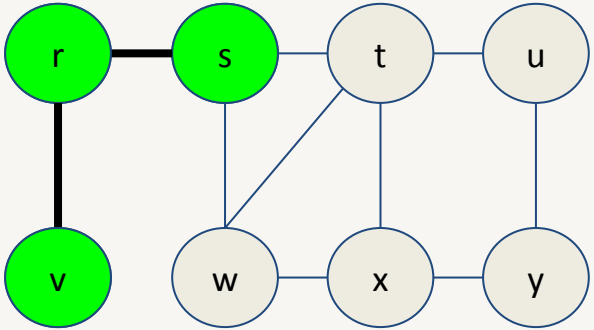
Start Time

2	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s r v

Stack

Visited

1	1	0	0	1	0	0	0
r	s	t	u	v	w	x	y

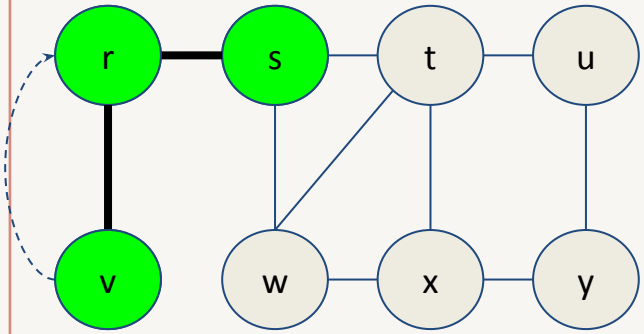
Start Time

2	1	0	0	3	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s r

Stack

Visited

1	1	0	0	1	0	0	0
r	s	t	u	v	w	x	y

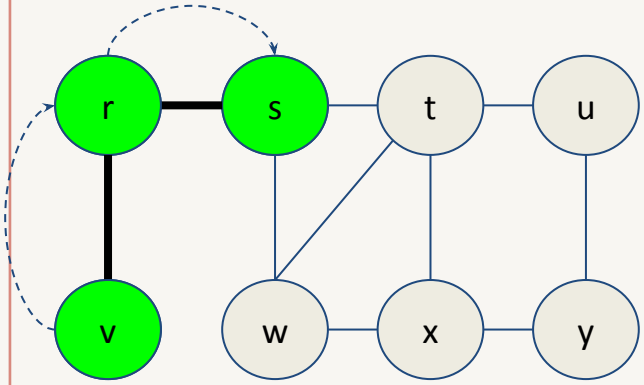
Start Time

2	1	0	0	3	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

1	1	0	0	1	0	0	0
r	s	t	u	v	w	x	y

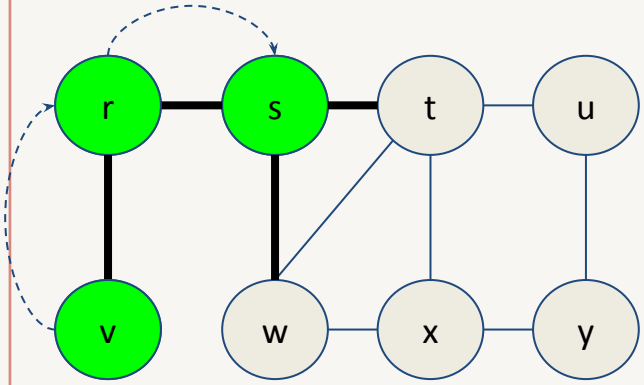
Start Time

2	1	0	0	3	0	0	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

1	1	0	0	1	0	0	0
r	s	t	u	v	w	x	y

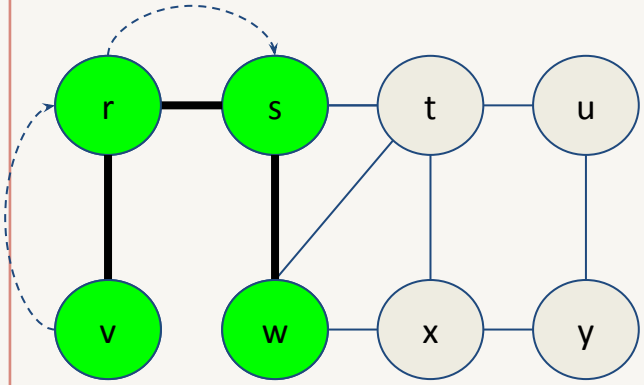
Start Time

2	1	0	0	3	0	0	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w

Stack

Visited

1	1	0	0	1	1	0	0
r	s	t	u	v	w	x	y

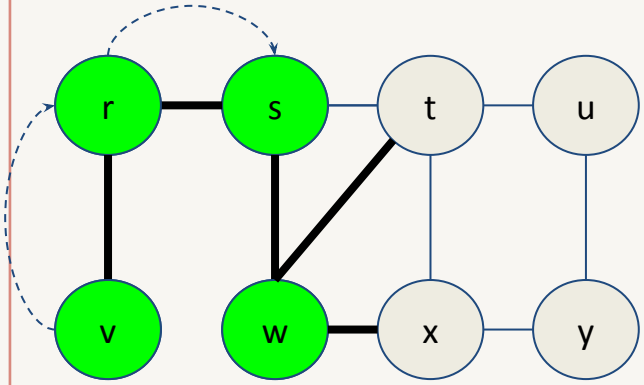
Start Time

2	1	0	0	3	6	0	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w

Stack

Visited

1	1	0	0	1	1	0	0
r	s	t	u	v	w	x	y

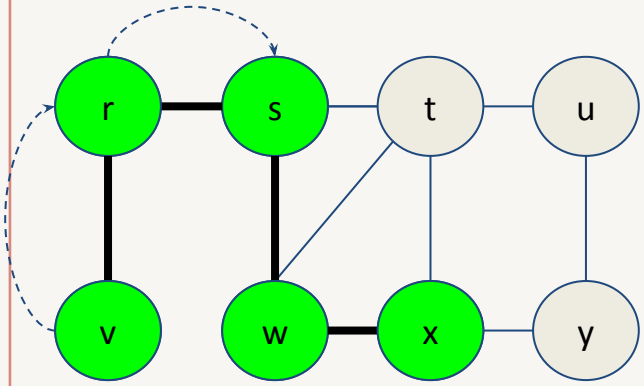
Start Time

2	1	0	0	3	6	0	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x

Stack

Visited

1	1	0	0	1	1	1	0
r	s	t	u	v	w	x	y

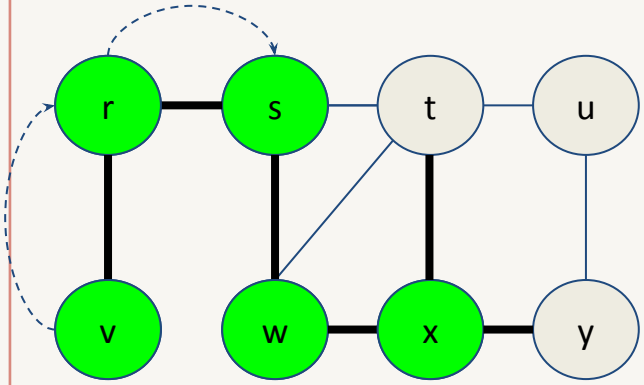
Start Time

2	1	0	0	3	6	7	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x

Stack

Visited

1	1	0	0	1	1	1	0
r	s	t	u	v	w	x	y

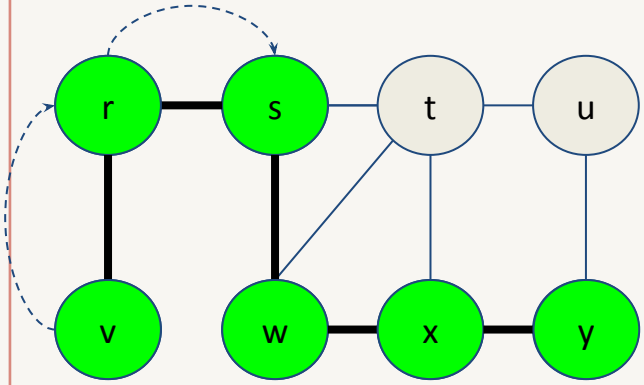
Start Time

2	1	0	0	3	6	7	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y

Stack

Visited

1	1	0	0	1	1	1	1
r	s	t	u	v	w	x	y

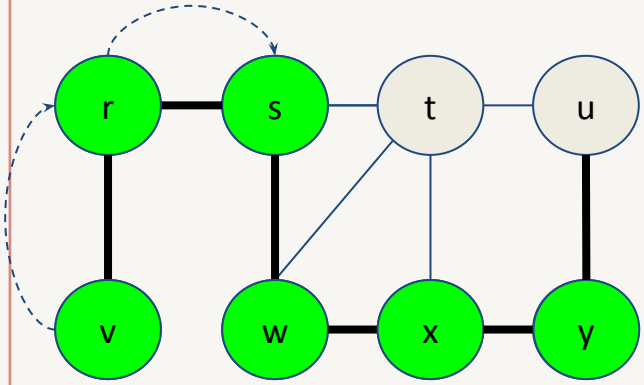
Start Time

2	1	0	0	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s w x y

Stack

Visited

1	1	0	0	1	1	1	1
r	s	t	u	v	w	x	y

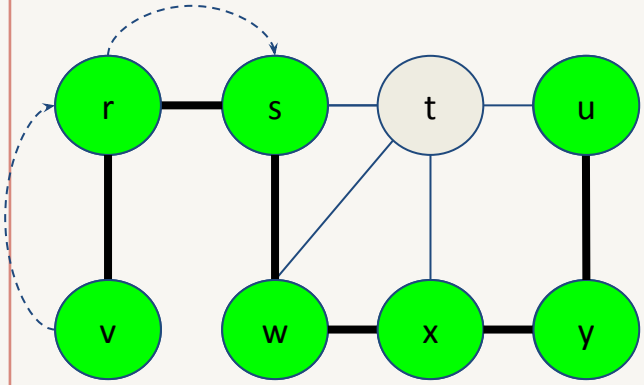
Start Time

2	1	0	0	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y	u

Stack

Visited

1	1	0	1	1	1	1	1
r	s	t	u	v	w	x	y

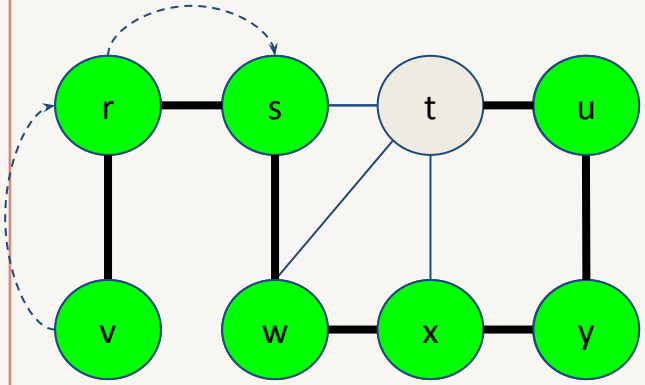
Start Time

2	1	0	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y	u

Stack

Visited

1	1	0	1	1	1	1	1
r	s	t	u	v	w	x	y

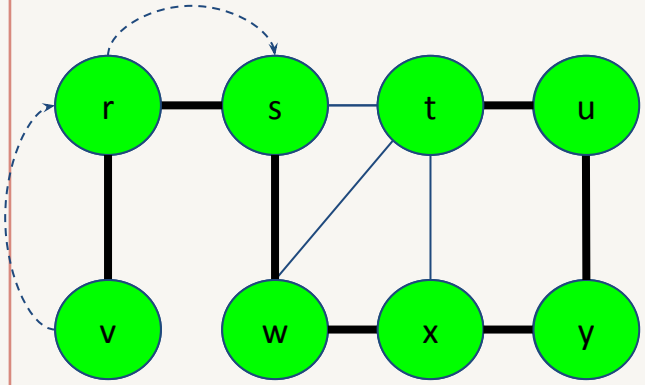
Start Time

2	1	0	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y	u	t

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

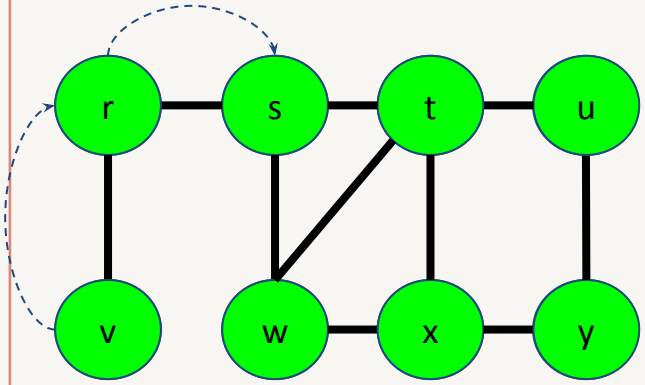
Start Time

2	1	9	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y	u	t

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

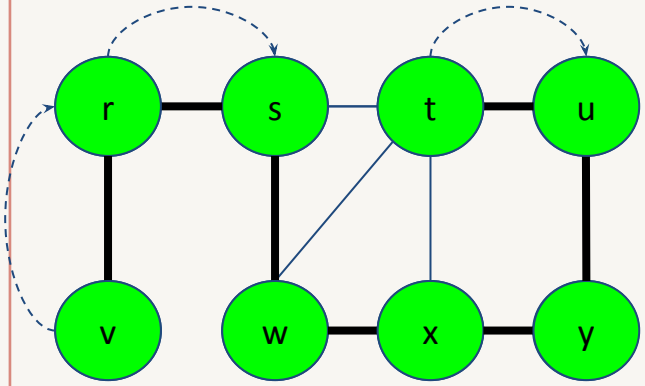
Start Time

2	1	9	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s w x y u

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

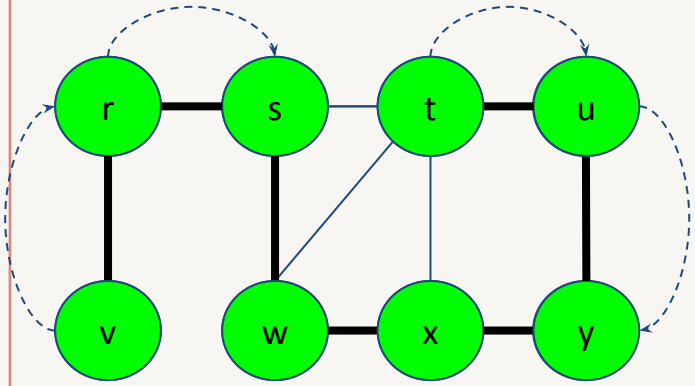
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s w x y

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

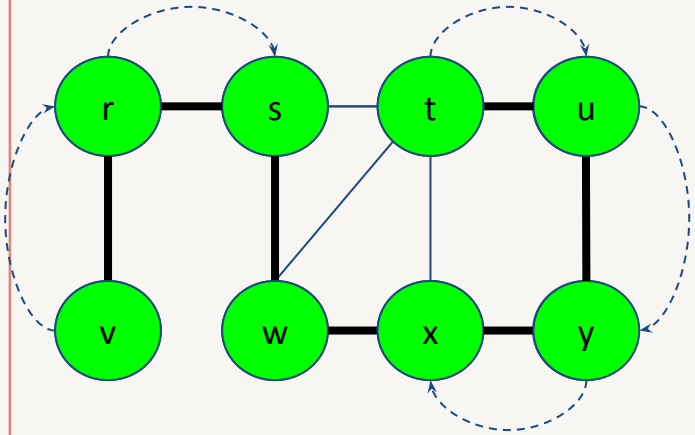
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	12	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s w x

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

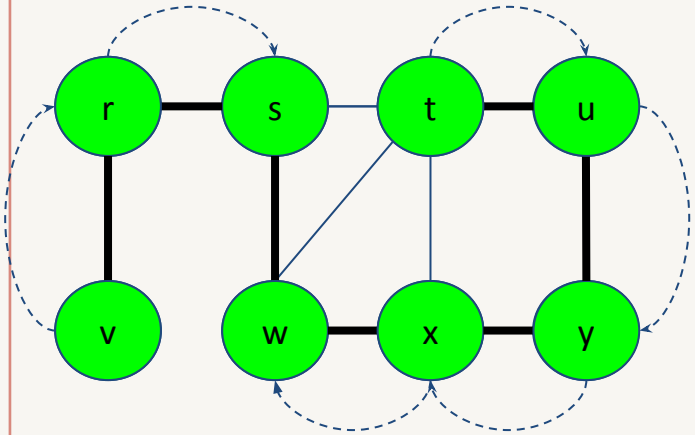
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	12	4	0	0	13
r	s	t	u	v	w	x	y

Simulation



s w

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

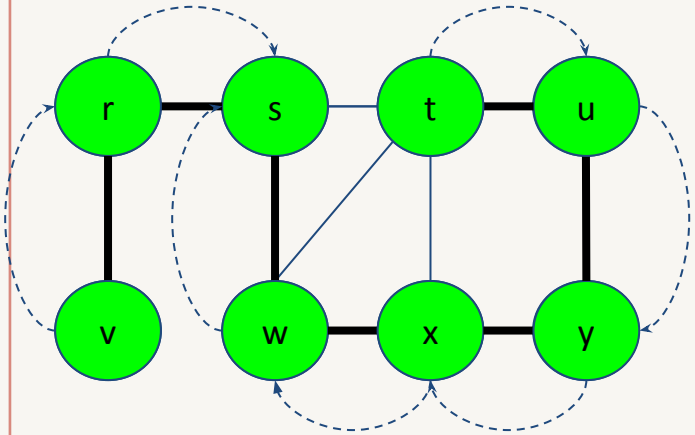
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	12	4	0	14	13
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

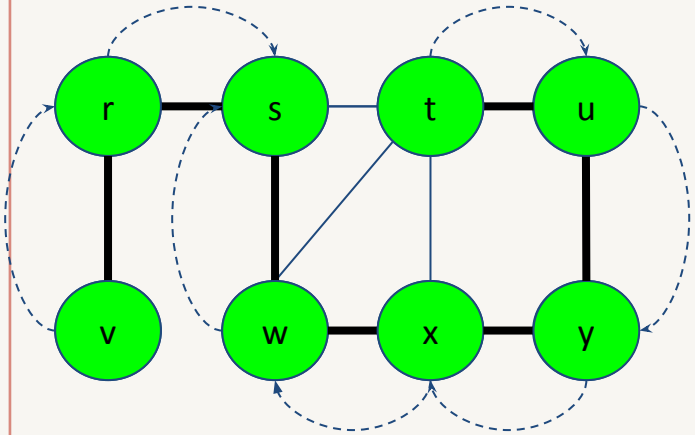
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	12	4	15	14	13
r	s	t	u	v	w	x	y

Simulation



Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

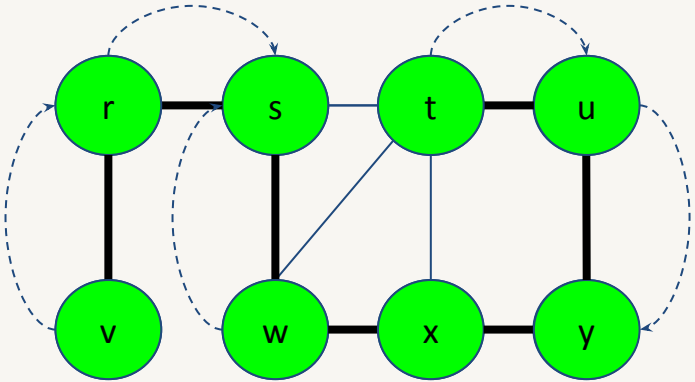
End Time

5	16	11	12	4	15	14	13
r	s	t	u	v	w	x	y

Stack

Since stack is empty, DFS has traversed all vertices!

Simulation



DFS traversal sequence will be,

s, r, v, w, x, y, u, t

Pseudocode

```
visited = [], start = [], end = [], time = 0

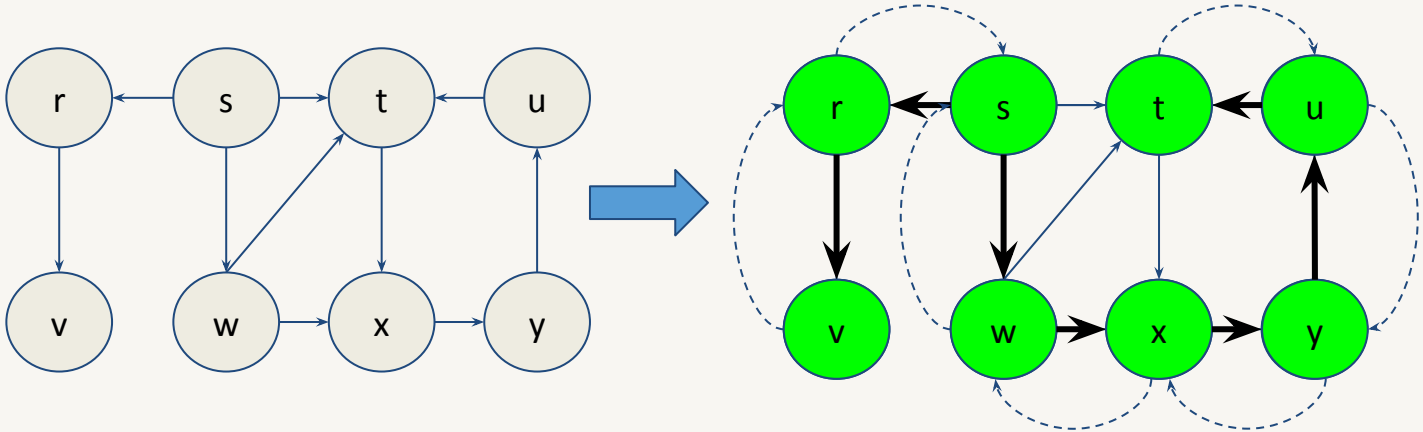
DFS(G, s)
    visited[s] = 1
    time = time + 1
    start[s] = time
    for each vertex v Є Adj[s]
        if visited[v] = 0
            DFS(G, v)
    time = time + 1
    end[s] = time
```

Properties

- **Time Complexity:** **$O(n + m)$** on a graph**,** **$O(b^d)$** on a search tree with branching factor b and depth d.
- **Space Complexity:** **$O(n)$** on a graph, **$O(bd)$** on a search tree.
- **Not *complete*** for infinite depth graphs or graphs with cycles. It may go infinitely deep along one branch and never backtrack, even though solution may exist somewhere. Complete only in **finite graphs with cycle detection or bounded depth**.
- **Not *optimal*** - may return a **suboptimal path** even if a shorter one exists.
- Good for **path existence**, **topological sorting**, **cycle detection**, **finding connected components** and traversing deep structures like **trees**.

Edge Classification

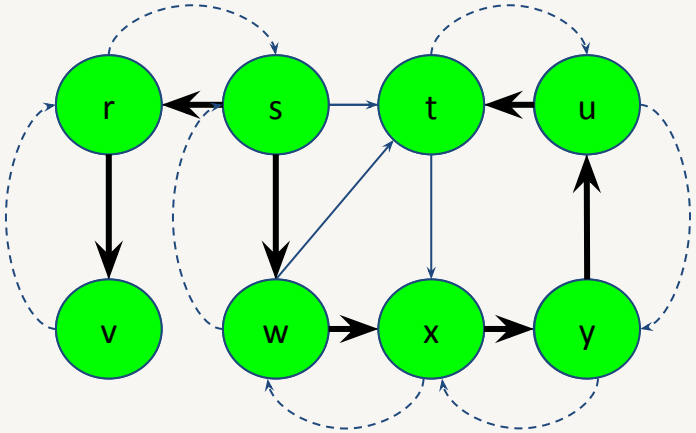
- Recall the DFS traversal of this **directed** graph



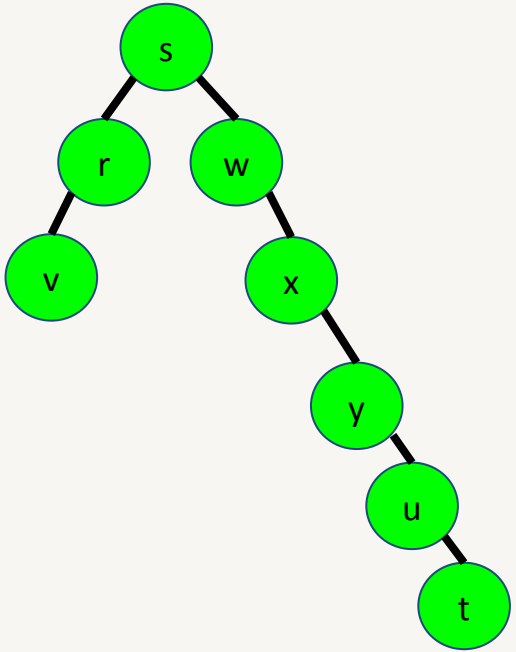
Types of Edge Classification

- Based on the **DFS Traversal of a particular graph**, we can classify the edges of that particular graph into **4 categories**
  - **Tree edge**
  - **Forward edge**
  - **Backward edge**
  - **Cross edge**

Tree Edge



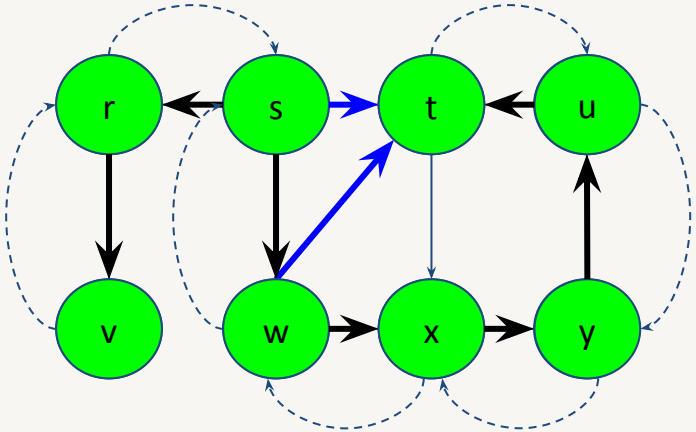




The edges (s, r), (r, v), (s, w), (w, x), (x, y), (y, u) and (u, t) are tree edges as these edges were used for the particular DFS Traversal.

Forward Edge

- An edge (u, v) is forward edge if (u, v) is not a tree edge and **ancestor(v) = u.**



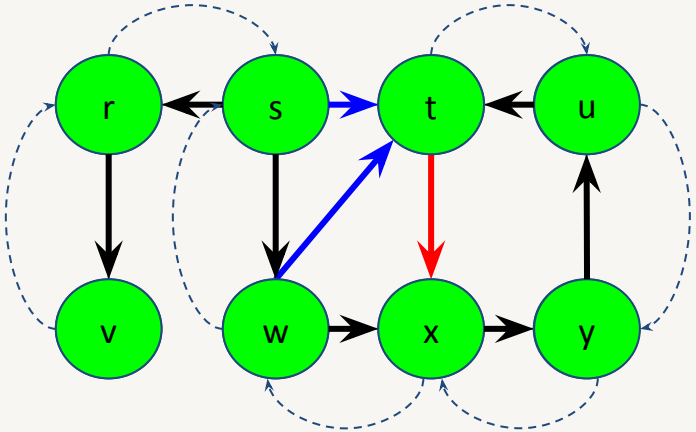
Here, consider the edge (w, t)

(w, t) -> not a tree edge
Ancestor(t) = w (Yes, if you backtrack from t, you will see w in the path). **Therefore, (w, t) is a forward edge.**

Similarly, (s, t) is also a forward edge!

Backward Edge

- An edge (u, v) is a backward edge if (u, v) is not a tree edge and **ancestor(u) = v.**



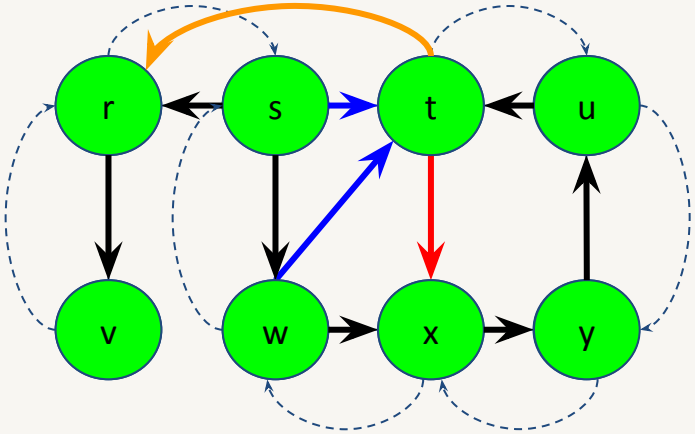
Here, consider the edge (t, x)

(t, x) -> not a tree edge
Ancestor(t) = x (Yes, if you backtrack from t, you will see x in the path). **Therefore, (t, x) is a backward edge.**

If any graph has any backward edge, that means cycle is present.
(t, x, y, u) is a cycle in the graph.

Cross Edge

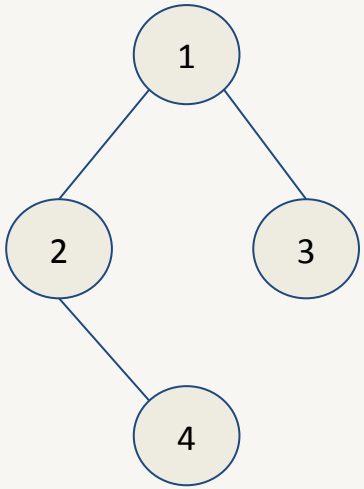
- An edge (u, v) is a cross edge if (u, v) is not a tree edge and **ancestor(u) ≠ v** or **ancestor (v) ≠ u**.



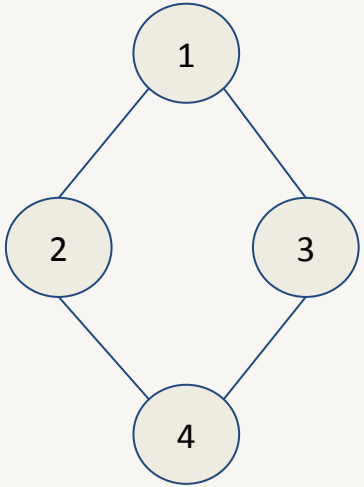
Here, consider if edge (t, r) existed

(t, r) -> not a tree edge
ancestor(t) ≠ r or ancestor(r) ≠ t (Yes, if you backtrack from t to r, you will not find r in the path. Again, if you backtrack from r to t, you will not find t in path). **Therefore, (t, r) is a cross edge.**

Cycle in an Undirected Graph

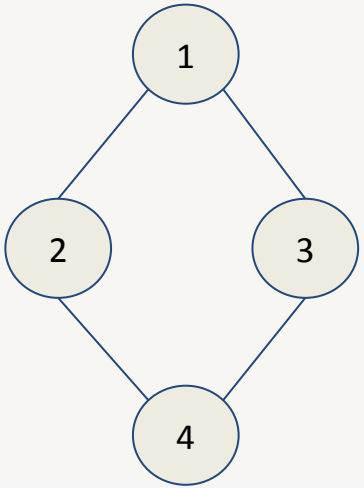


No cycle



Cycle

Cycle Detection using DFS



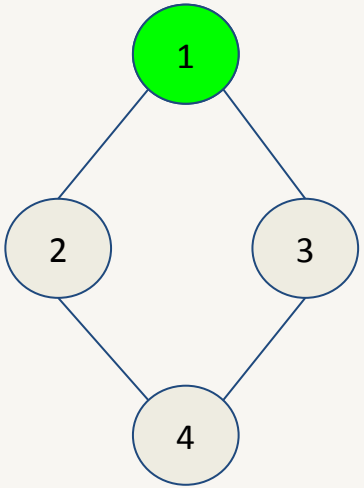
Visited

0	0	0	0
1	2	3	4

Parent

0	0	0	0
1	2	3	4

Cycle Detection using DFS



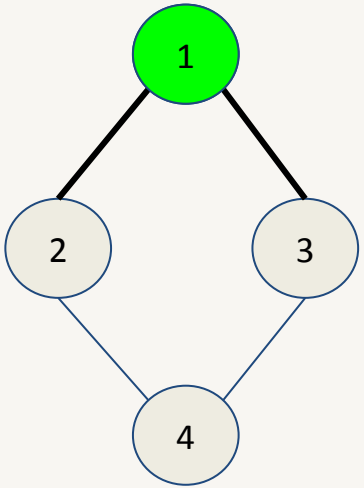
Visited

1	0	0	0
1	2	3	4

Parent

-1	0	0	0
1	2	3	4

Cycle Detection using DFS



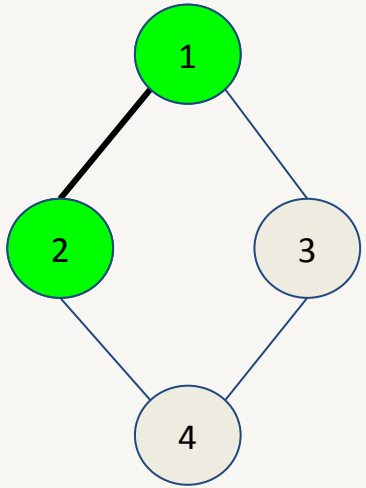
Visited

1	0	0	0
1	2	3	4

Parent

-1	0	0	0
1	2	3	4

Cycle Detection using DFS



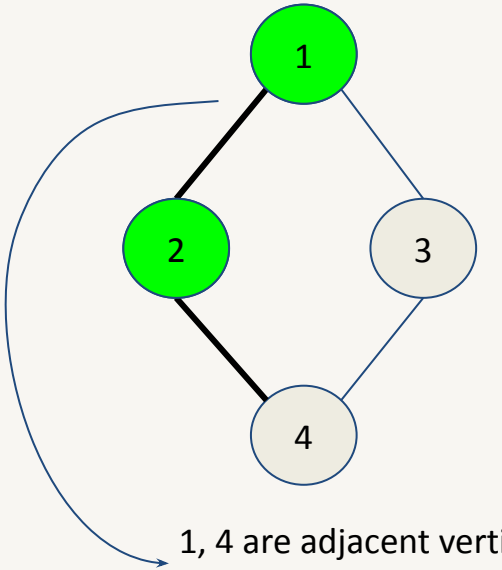
Visited

1	1	0	0
1	2	3	4

Parent

-1	1	0	0
1	2	3	4

Cycle Detection using DFS



Visited

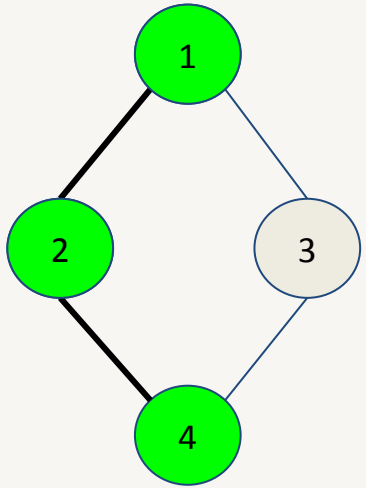
1	1	0	0
1	2	3	4

Parent

-1	1	0	0
1	2	3	4

1, 4 are adjacent vertices of 2. However, 1 is already visited and 1 is also parent of 2. Therefore, it is not a cycle and we will visit 4 since it is unvisited yet.

Cycle Detection using DFS



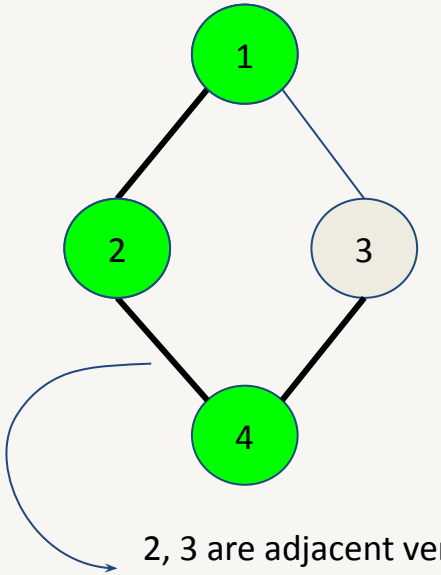
Visited

1	1	0	1
1	2	3	4

Parent

-1	1	0	2
1	2	3	4

Cycle Detection using DFS



Visited

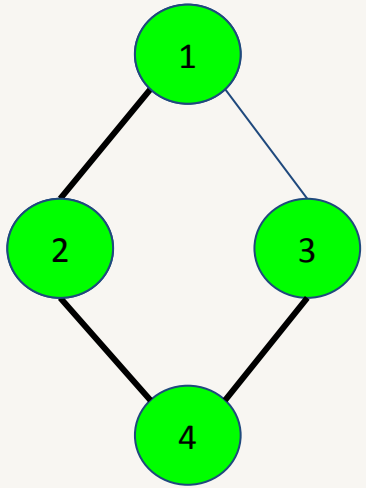
1	1	0	1
1	**2**	**3**	**4**

Parent

-1	1	0	2
1	**2**	**3**	**4**

2, 3 are adjacent vertices of 4. However, 2 is already visited and 2 is also parent of 4. Therefore, it is not a cycle and we will visit 3 as it is unvisited yet.

Cycle Detection using DFS



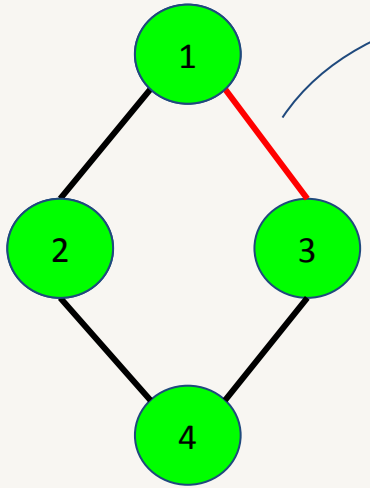
Visited

1	1	1	1
1	**2**	**3**	**4**

Parent

-1	1	4	2
1	**2**	**3**	**4**

Cycle Detection using DFS



Visited

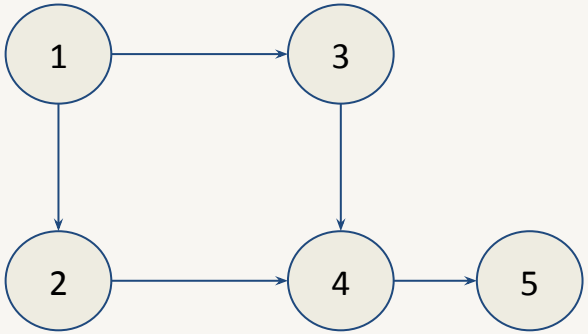
1	1	1	1
1	**2**	**3**	**4**

Parent

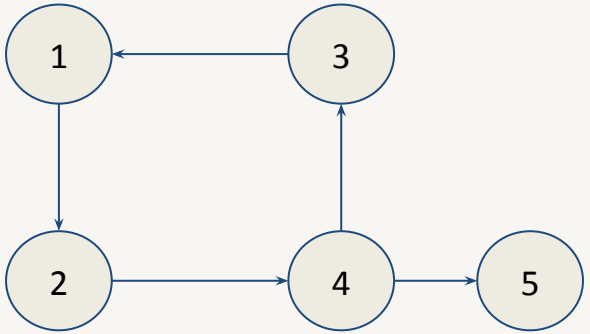
-1	1	4	2
1	**2**	**3**	**4**

4, 1 are adjacent vertices of 3. However, 4 is already visited and 4 is also parent of 3. Therefore, it is not a cycle. We will then check 1 and since 1 is already visited, we will check whether 1 is parent of 3. **Since parent of 3 is 4, therefore, there exists a cycle!**

Cycle in a Directed Graph

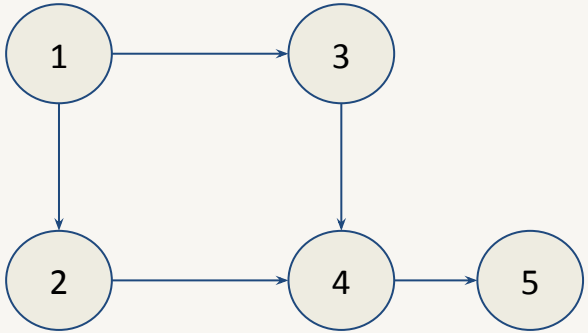


No cycle



Cycle

Cycle Detection using DFS



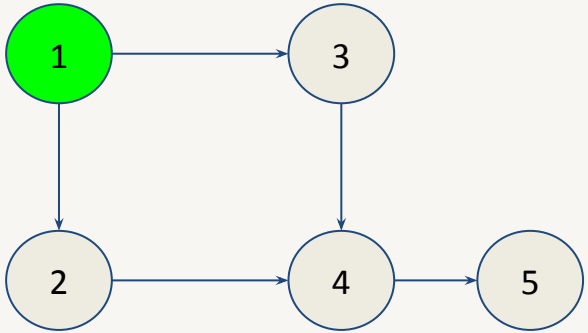
Visited

0	0	0	0	0
1	2	3	4	5

Parent

N	N	N	N	N
1	2	3	4	5

Cycle Detection using DFS



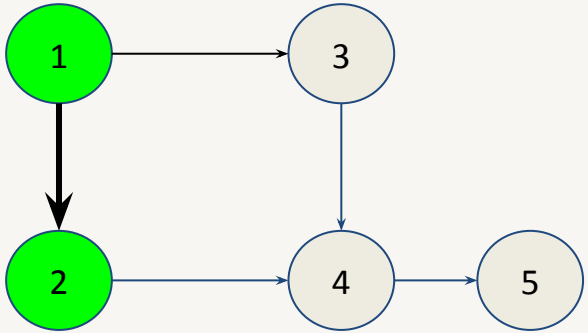
Visited

1	0	0	0	0
1	2	3	4	5

Parent

-1	N	N	N	N
1	2	3	4	5

Cycle Detection using DFS



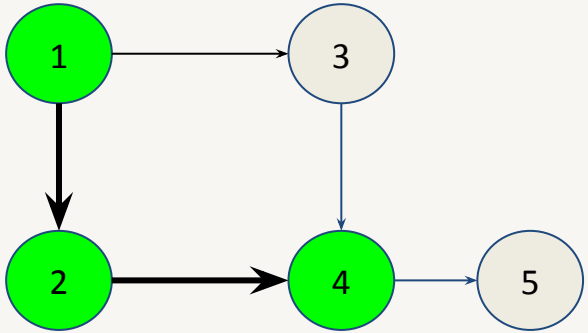
Visited

1	1	0	0	0
1	2	3	4	5

Parent

-1	1	N	N	N
1	2	3	4	5

Cycle Detection using DFS



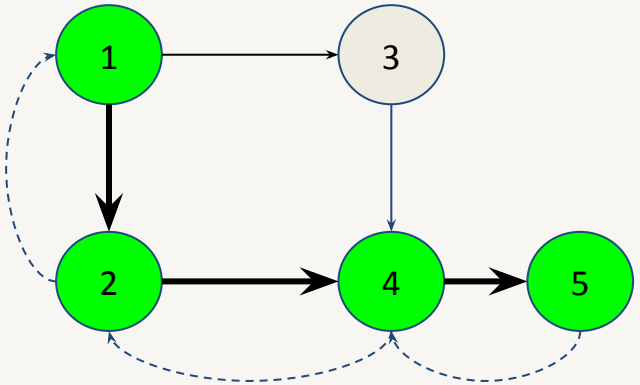
Visited

1	1	0	1	0
1	2	3	4	5

Parent

-1	1	N	2	N
1	2	3	4	5

Cycle Detection using DFS



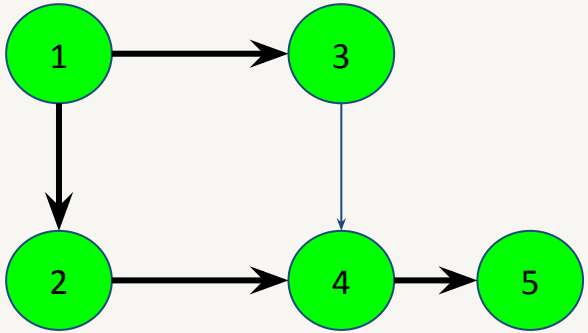
Visited

1	1	0	1	1
1	2	3	4	5

Parent

-1	1	N	2	4
1	2	3	4	5

Cycle Detection using DFS



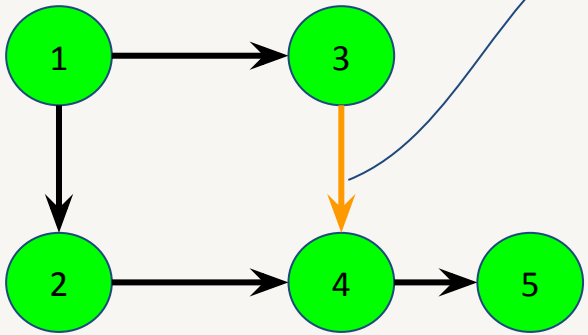
Visited

1	1	1	1	1
1	2	3	4	5

Parent

-1	1	1	2	4
1	2	3	4	5

Cycle Detection using DFS



Visited

1	1	1	1	1
1	2	3	4	5

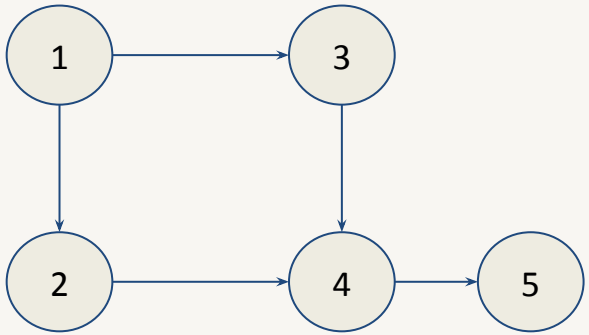
Parent

-1	1	1	2	4
1	2	3	4	5

3 has adjacent vertex 4, which has already been visited. So according to previous way of cycle detection in an undirected graph, we will check parent of 3 which is 1 and not 4. This will return True which means a cycle is present. However, there are no cycles in this graph! Workaround?

Cycle Detection

We will not follow the approach used to detect cycle in an undirected graph. Instead, we will keep a list/stack named currentPath and will check whether the vertex exists in currentPath or not.

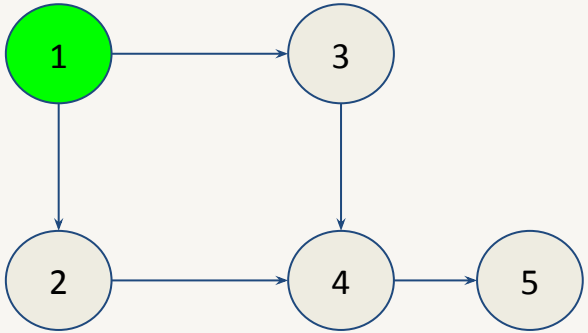


Visited

0	0	0	0	0
1	2	3	4	5

currentPath

Cycle Detection



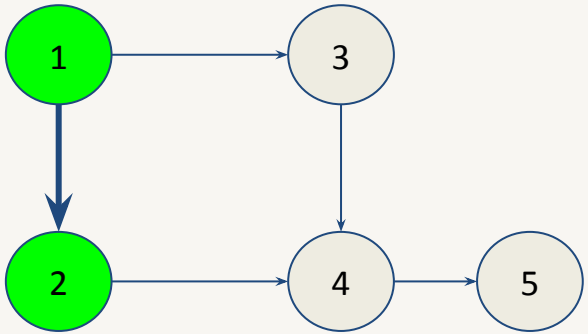
Visited

1	0	0	0	0
1	**2**	**3**	**4**	**5**

1

currentPath

Cycle Detection



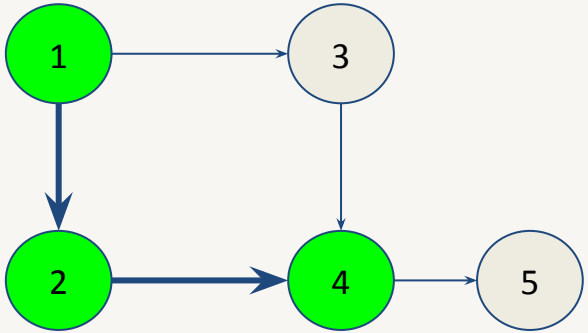
Visited

1	1	0	0	0
1	**2**	**3**	**4**	**5**

1 2

currentPath

Cycle Detection



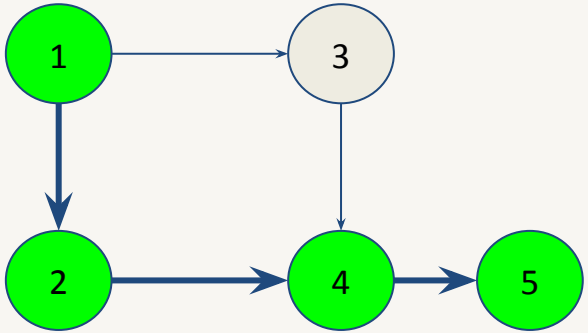
Visited

1	1	0	1	0
1	**2**	**3**	**4**	**5**

1 2 4

currentPath

Cycle Detection



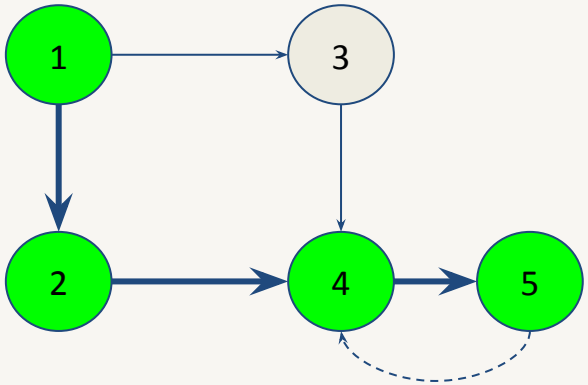
Visited

1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2 4 5

currentPath

Cycle Detection



Visited

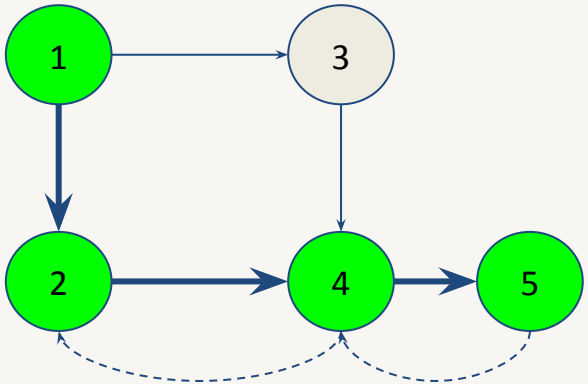
1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2 4

currentPath

As we backtrack from 5 to 4, we will remove 5 from Current Path.

Cycle Detection



Visited

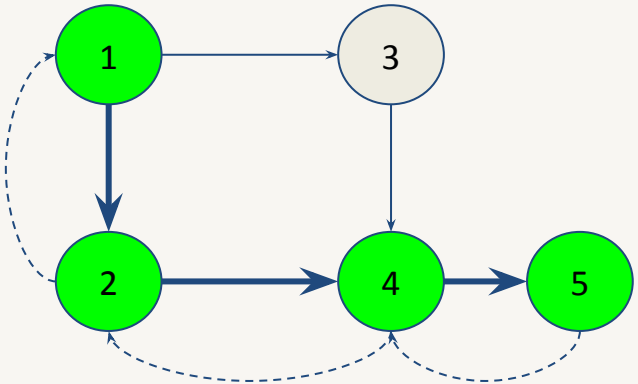
1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2

currentPath

As we backtrack from 4 to 2, we will remove 4 from Current Path.

Cycle Detection



Visited

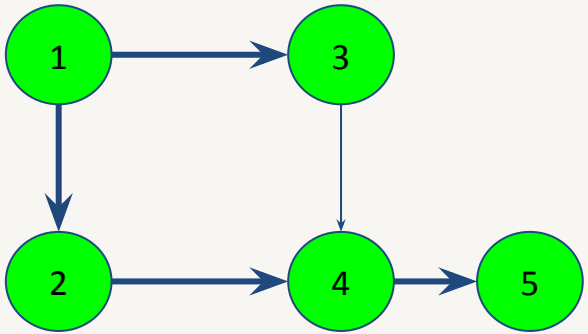
1	1	0	1	1
1	**2**	**3**	**4**	**5**

1

currentPath

As we backtrack from 2 to 1, we will remove 2 from Current Path.

Cycle Detection



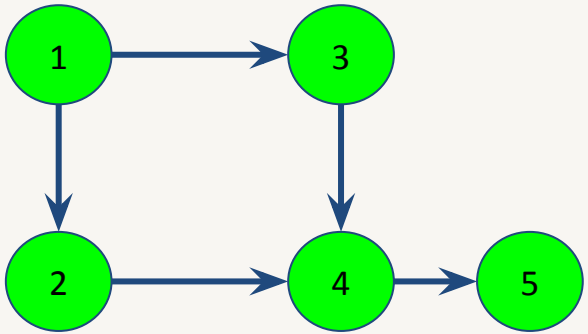
Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

1 3

currentPath

Cycle Detection



Visited

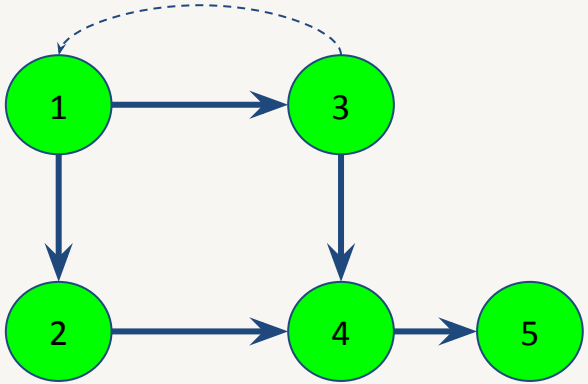
1	1	1	1	1
1	**2**	**3**	**4**	**5**

1 3

currentPath

3 has adjacent vertex 4. However, 4 is already visited. So we will check whether 4 exists in currentPath or not. **Since it does not exist in currentPath, no cycle is present in the above graph.**

Cycle Detection



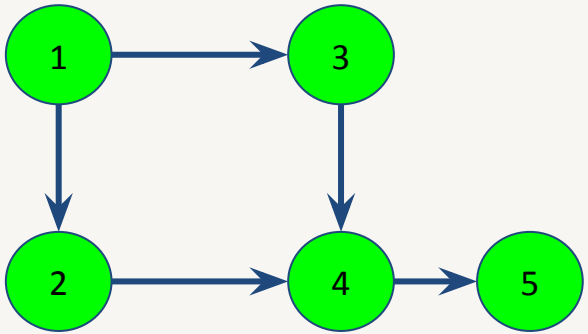
Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

1

currentPath

Cycle Detection

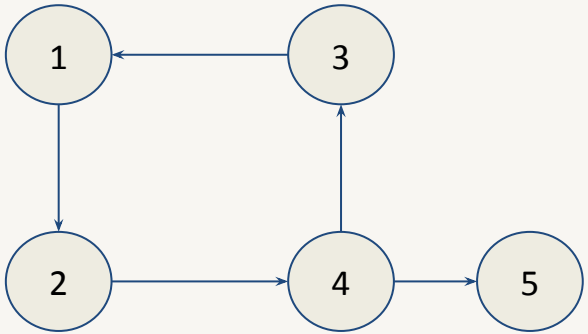


Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

currentPath

Cycle Detection

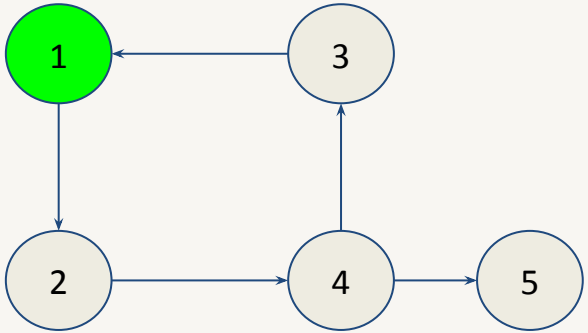


Visited

0	0	0	0	0
1	2	3	4	5

currentPath

Cycle Detection



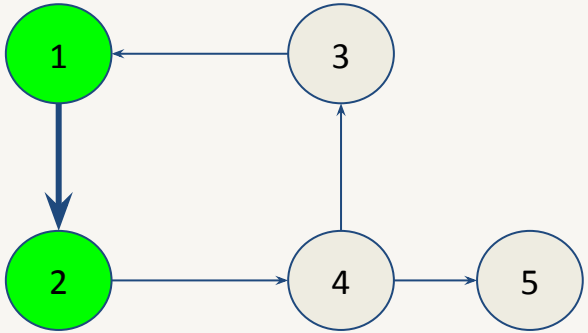
Visited

1	0	0	0	0
1	**2**	**3**	**4**	**5**

1

currentPath

Cycle Detection



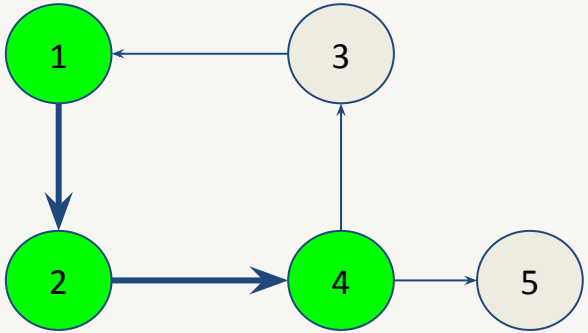
Visited

1	1	0	0	0
1	**2**	**3**	**4**	**5**

1 2

currentPath

Cycle Detection



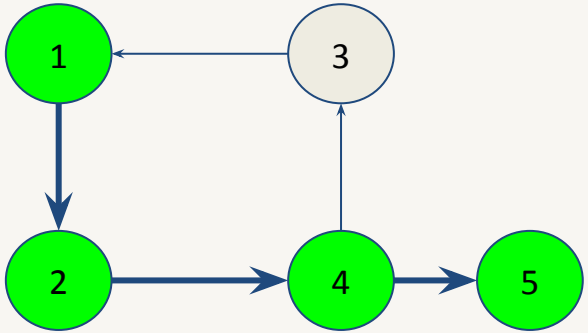
Visited

1	1	0	1	0
1	**2**	**3**	**4**	**5**

1 2 4

currentPath

Cycle Detection



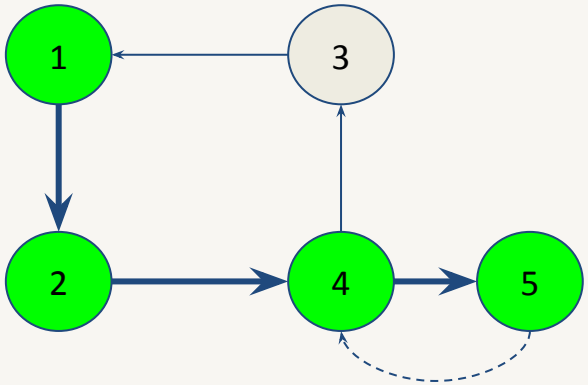
Visited

1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2 4 5

currentPath

Cycle Detection



Visited

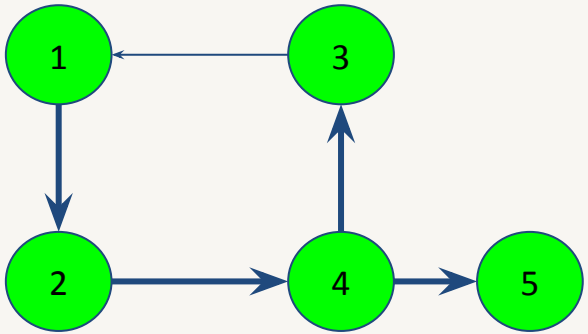
1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2 4

currentPath

As we backtrack from 5 to 4, we will remove 5 from Current Path.

Cycle Detection



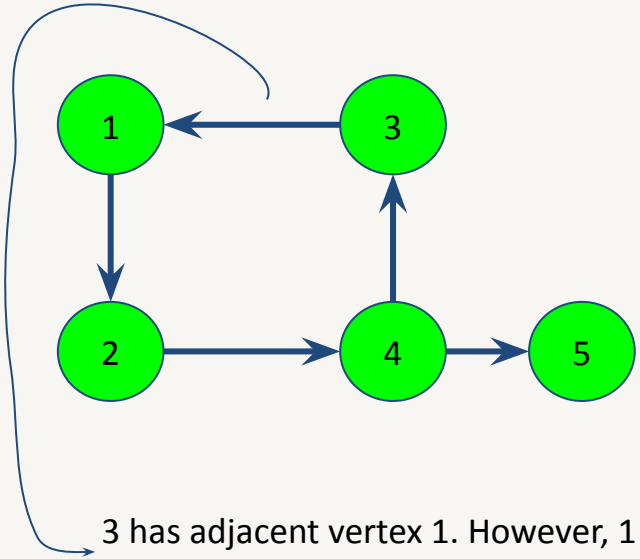
Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

1 2 4 3

currentPath

Cycle Detection



Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

1 2 4 3

currentPath

3 has adjacent vertex 1. However, 1 is already visited. So we will check whether 1 exists in currentPath or not. **Since it exists in currentPath, therefore, cycle is present in the above graph.**